Day 50 – Content Enricher & Poll Enrich

Class date: 7th September 2026 | Module: Integration Patterns (Message Transformation)

Good accuracy today – nothing needed correcting. Everything in your notes checked out against SAP's own documentation: the Combine vs. Enrich algorithms, the Path to Node/Key Element fields, the Content Enricher's XML-only limitation, and the three Poll Enrich algorithms (Replace, Combine XML, Concatenate) all matched exactly. Below, I've organized everything clearly and added a couple of extra "why this matters" details you'll find useful in interviews.

1. What is a Content Enricher? (start here first)

Whenever you want to **enhance** something – i.e. add **additional data** to an already-existing payload – you use a **Content Enricher**. In short: **Content Enricher = enhancing/enriching the existing payload/message** with more information.

Simple example: a Source system sends basic Employee Information, but you also need some **additional information** that lives on a different, **Target** system.

Scenario walked through in class:

- Employee Information is fetched via an API call from the **Source** – but this only returns the **Employee ID**, not the full details.
- So a **2nd API call** is needed: fetch → combine → send. This 2nd call fetches the full Employee Information, based on the Employee ID obtained from the first call.

Example used in the demo – Northwind API: the Source only sends the incoming Employee ID as input payload:

```
1
```

Based on this EmpID, CPI needs to **get the full Employee Information** (via the Northwind API), then **combine** that information with the input payload, and **send** the combined result to the target. This overall pattern – existing payload + additional API call + combine + send – is exactly what the **Content Enricher** does.

2. iFlow structure for a Content Enricher

Call palette location: **External** → **Content Enricher.**

Flow shape: **Sender** → **Content Enricher**, with a **Receiver** participant connected above it via an **OData** call (Northwind API in the demo). The Content Enricher step **reads the data from the Receiver** (the external lookup call) and merges it with what came in from the Sender.

Two payloads get combined: (1) the **Input payload** (Message 1, from the Sender), and (2) the result of the **2nd API call** (Employee Details, from the Receiver/lookup system).

Note: for enriching an existing payload/message with an API call, that additional data typically needs to be in **XML** format for the Content Enricher to merge it correctly.

3. Alternative approach: Content Modifier + Request-Reply

This same enrichment process can also be done using a Content Modifier instead of a dedicated Content Enricher step.

How: in the Content Modifier, create a **Property** that stores the **Input payload** (using expression **${body}**, as a **String**). The flow continues: **Content Modifier** → **Request-Reply** (which makes the OData call) → **a second Content Modifier**, which then **combines the data** – pulling the originally-stored Input payload property back in alongside the fresh Request-Reply response.

```
Sender -> Content Modifier (store input payload as property) -> Request-Reply (OData call) -> Content Modifier #2 (combine stored property + new response) -> Receiver
```

Example expression structure used to rebuild the combined message:

```
${property.InputPayload} ${body}
```

Scenario summary: Incoming payload → stored in a property variable inside the first Content Modifier → API call (OData) to get related information → append/combine both pieces together in the second Content Modifier → send.

4. Content Enricher – the dedicated pattern

Alternative to building it manually with Content Modifiers: use the dedicated **Content Enricher** step directly, connected via a **Request-Reply**-style call to the external system (e.g. OData call to get Employees).

Configuration – when you click on the Content Enricher palette function, a dropdown appears for **Aggregation Algorithm**, with two options:

Algorithm	What it does
Combine (append)	Combines the two payloads *as-is* – simply places one message after another (top & bottom), with **no matching/merge rules** applied.
Enrich	Merges the two messages based on a **respective/common ID** – i.e. it looks up and matches records using a key field shared between the original and lookup messages.

Enrich, in detail: you configure the matching using **Path to Node** and **Key Element**, for both the **Original Message** and the **Lookup Message**:

	Path to Node	Key Element
Original Message	/EmployeeInfo/EmpDetails	EmpID
Lookup Message	/Employees/Employee	EmployeeID

CPI matches the **EmpID** in the Original Message against the **EmployeeID** in the Lookup Message, and merges the two records together wherever those values match.

5. Related entities & a real-time scenario

Extending the example – Orders related to an Employee:

	Path to Node	Key Element
Lookup Message (Orders)	/Orders	EmployeeID

This returns **all Orders related to the Employee ID(s)** present in the input payload.

Real-time scenario given in class: S/4HANA sends an **Event Mesh** notification for a new **Purchase Order**. That message only contains the **Vendor** information at first – so CPI makes an **API call** to fetch the related Vendor details, **appends** them to the original message, and **sends** the enriched result onward.

Which adapters can be used as the lookup call for a Content Enricher? OData, SuccessFactors, SOAP – but note: **Content Enricher only supports XML data, not JSON.** This matters in practice: an **OData V4** source returns **JSON only** (V4 doesn't support XML at all, as covered on Day 49), so a V4-based lookup would **not** work directly with a Content Enricher – you'd need OData V2 (or another XML-based adapter/SOAP) for the lookup call instead.

Why this XML-only limitation exists: the **Content Enricher only supports API-based (receiver-type) connections** – adapters that CPI actively calls out to and gets an XML response back from (HTTP, OData, SOAP). It cannot use a **polling/sender-type** adapter like SFTP as its lookup source – that's exactly the gap **Poll Enrich** (Section 6) is built to fill.

6. Poll Enrich

Why a separate pattern is needed: the Content Enricher only supports API-based connections. So, **if you want to pick a file from an SFTP Sender in the middle of the iFlow, you use Poll Enrich instead.**

Remember – very, very important: Poll Enrich reads the file from a Sender participant (a polling-type adapter, e.g. SFTP), not from a Receiver call like the Content Enricher does. This is the core distinction between the two patterns.

Flow shape: **Sender (main trigger, e.g. HTTPS)** → **Poll Enrich** → **Receiver**, with a second **Sender participant** attached directly to the Poll Enrich step via **SFTP** – this second Sender is what actually gets *polled* mid-flow for a file.

```
Main Sender (HTTPS) -> Poll Enrich <- SFTP Sender (polled for a file) | v Receiver
```

Poll Enrich – Processing tab: Aggregation Algorithm

Algorithm	What it does
Replace	Simply replaces the original message (payload and headers) with the new one fetched from the SFTP server.
Combine XML	Combines the polled message with the original message in multi-mapping format – used when both the original and polled messages are XML.
Concatenate	Adds the polled message onto the existing message – useful when working with plain strings rather than structured XML.

Also available: a **"Throw Exception if no Message Found"** checkbox – if no file is available on the SFTP server at poll time, the step raises an exception instead of silently continuing with nothing.

Quick-recall summary

- **Content Enricher** = enhance an existing payload with data from an **API-based (receiver-type)** lookup call – **Combine** (append as-is) or **Enrich** (merge by matching Path to Node + Key Element).

- The same result can be built manually with **Content Modifier (store payload as property)** → **Request-Reply** → **Content Modifier (recombine)**.
- Content Enricher supports **only XML** – incompatible with a JSON-only OData V4 lookup source.
- **Poll Enrich** = the pattern for picking up a file from a **polling/Sender-type** adapter (e.g. SFTP) mid-flow – Content Enricher cannot do this.
- Poll Enrich's 3 Aggregation Algorithms: **Replace, Combine XML, Concatenate.**